



الجامعة الإسلامية العالمية ماليزيا
INTERNATIONAL ISLAMIC UNIVERSITY MALAYSIA
يُونَيْتِي إِسْلَامُ إِنْتَارَا بَعْثَا مِلْدِيَا

Garden of Knowledge and Virtue

**MECHATRONIC SYSTEM INTEGRATION
(MCTA 3203)**

SEMESTER 2, 2025/2026

**EXPERIMENT 8:
REMOTE MONITORING SYSTEM AND CONTROL VIA BLUETOOTH**

(WEEK 9 REPORT)

SECTION 1

GROUP 8

MATRIC NO.	NAME
2416289	MUHAMMAD IRFAN HAKIMI BIN MOHD RIZUAN
2128653	MUHAMMAD NAJMI BIN MOHD AZAM
2120279	MUHAMMAD SHAHABUDIN BIN ISHAK

SUBMISSION DATE: 13 MAY 2026

ABSTRACT

This report presents the development of a Bluetooth-based remote monitoring and control system using an ESP32 microcontroller. The experiment focuses on wireless communication between the ESP32 and a smartphone terminal application. Sensor data is transmitted from the ESP32 to the smartphone, while control commands are sent from the smartphone back to the ESP32. The system is designed to demonstrate the basic concept of Internet of Things communication, where a microcontroller can collect physical data and allow remote control without a wired serial connection.

The Week 9 task required the system to include one sensor and one servo motor. The sensor provides real-time data to the smartphone through Bluetooth communication, while the servo responds to commands entered through the smartphone terminal. This implementation shows how Bluetooth can be used for short-range monitoring, control, troubleshooting, and simple automation in mechatronic systems.

TABLE OF CONTENTS

CONTENTS	PAGE
ABSTRACT	2
INTRODUCTION	4
OBJECTIVES	4
MATERIAL AND EQUIPMENT	4
THEORY	4
EXPERIMENT SETUP	5
METHODOLOGY	5
DATA COLLECTION	6
TASK 1: BLUETOOTH IN IOT APPLICATIONS	7
TASK 2: SENSOR MONITORING AND SERVO CONTROL	7
DISCUSSION	8
CONCLUSION	8
RECOMMENDATIONS	8
REFERENCES	9
APPENDICES	9
Appendix A: Program Listing	9
Appendix B: Experimental Figures	10
Appendix C: Acknowledgement	10
Appendix C: Declaration	11

INTRODUCTION

Bluetooth communication is commonly used in embedded and mechatronic systems because it allows devices to exchange data wirelessly over a short distance. In this experiment, the ESP32 is used because it has built-in Bluetooth capability, making it suitable for creating a simple wireless interface between a microcontroller and a smartphone. The smartphone acts as a user interface that can display sensor readings and send control commands to the ESP32.

The main purpose of this experiment is to understand how Bluetooth communication can be applied to remote monitoring and control. Instead of using the Arduino Serial Monitor through a USB cable, data is sent through a Bluetooth terminal application. This creates a more flexible system because the user can monitor sensor values and control an actuator from a paired mobile device. The experiment also demonstrates the relationship between sensor input, wireless data transmission, and actuator output in an integrated mechatronic system.

OBJECTIVES

1. To establish Bluetooth communication between an ESP32 and a smartphone.
2. To transmit sensor readings from the ESP32 to a Bluetooth terminal application.
3. To receive control commands from the smartphone and process them using the ESP32.
4. To control a servo motor based on commands sent through Bluetooth.
5. To explain how Bluetooth communication can be incorporated into Internet of Things applications.

MATERIAL AND EQUIPMENT

- ESP32 development board
- One sensor module, such as DHT22 temperature and humidity sensor, potentiometer, ultrasonic sensor, or light sensor
- Servo motor
- Smartphone with Bluetooth terminal application
- Power supply for ESP32 and external servo supply if required
- Breadboard
- Jumper wires
- USB cable
- Arduino IDE with required ESP32 board support and libraries

THEORY

The ESP32 supports Bluetooth wireless communication using a 2.4 GHz radio frequency. Bluetooth allows data packets to be exchanged between paired devices without the need for a physical cable. In a simple embedded system, the ESP32 can be configured as a Bluetooth serial device so that a smartphone can send and receive text data through a terminal application.

Bluetooth serial communication works similarly to a wired UART serial connection from the user perspective. The ESP32 sends data in the form of text strings, and the smartphone

terminal displays the received information. At the same time, the smartphone can transmit characters or commands back to the ESP32. These commands can then be interpreted by the microcontroller to control an output device, such as an LED, relay, or servo motor.

In this experiment, Bluetooth is used for two-way communication. The first direction is monitoring, where sensor values are read by the ESP32 and transmitted to the smartphone. The second direction is control, where a command from the smartphone changes the position of a servo motor. This arrangement represents a basic remote monitoring and control system, which is a common foundation for Internet of Things and smart automation applications.

EXPERIMENT SETUP

Hardware Setup

The ESP32 was prepared as the main controller for the Bluetooth monitoring and control system. The selected sensor was connected to the appropriate input pin of the ESP32. If a DHT22 sensor is used, the data pin should be connected to a digital GPIO pin, while VCC and GND should be connected to the ESP32 supply and ground. If an analog sensor is used, its signal output should be connected to an ADC-capable GPIO pin on the ESP32.

The servo motor was connected to a PWM-capable GPIO pin of the ESP32. The signal pin of the servo receives the control pulse from the ESP32, while the power and ground pins must be connected correctly. For stable operation, the servo should be powered using a suitable external supply when necessary, and the ground of the external supply must be connected to the ESP32 ground.

Software Setup

6. Open the Arduino IDE and select the correct ESP32 board and COM port.
7. Install the required library for the selected sensor, if applicable.
8. Prepare the Bluetooth serial program for the ESP32.
9. Set the Bluetooth device name so that it can be identified from the smartphone.
10. Install a Bluetooth terminal application on the smartphone.
11. Pair the smartphone with the ESP32 Bluetooth device.
12. Open the terminal application and connect to the ESP32.
13. Observe the transmitted sensor readings and test servo control commands.

METHODOLOGY

Circuit Assembly

The circuit was assembled by connecting the sensor and servo motor to the ESP32. The sensor served as the input device, while the servo motor served as the output actuator. All ground connections were joined together to ensure a common reference voltage. The wiring was checked carefully before uploading the program to prevent incorrect polarity, unstable readings, or servo malfunction.

Bluetooth Pairing Procedure

After the program was uploaded to the ESP32, the Bluetooth device name became visible from the smartphone. The smartphone was paired with the ESP32 through the Bluetooth settings or directly through the Bluetooth terminal application. Once the connection was successful, the terminal application functioned as a wireless serial monitor.

Programming Logic

The program was designed to perform two main operations. First, the ESP32 continuously reads the sensor value and sends the data to the smartphone through Bluetooth. Second, the ESP32 checks whether a command has been received from the smartphone. If a valid command is received, the ESP32 changes the servo motor position according to the command. This creates a closed interaction between user input, controller processing, sensor feedback, and actuator response.

Testing Procedure

14. Power the ESP32 and confirm that the Bluetooth device is visible on the smartphone.
15. Connect the smartphone terminal application to the ESP32.
16. Observe whether the sensor reading is displayed continuously on the terminal.
17. Send the first command to rotate the servo to its initial or low-angle position.
18. Send the second command to rotate the servo to a middle or high-angle position.
19. Repeat the test several times to verify that the received commands consistently control the servo.
20. Record the sensor reading, command sent, and servo response in the data collection table.

DATA COLLECTION

The following table is prepared for recording the observed system response during the experiment. The exact sensor reading should be filled in based on the reading displayed on the smartphone terminal during testing.

Test No.	Sensor Reading Displayed on Smartphone	Command Sent from Smartphone	Servo Response
1	[A]	Command for low-angle position	Servo moves to low-angle position
2	[B]	Command for middle position	Servo moves to middle position
3	[C]	Command for high-angle position	Servo moves to high-angle position
4	[D]	No valid command	Servo remains in previous position

Based on the testing procedure, the expected output is that the smartphone terminal receives the sensor value from the ESP32 and displays it as readable text. When the user sends a valid command, the ESP32 interprets the command and changes the servo position.

This confirms that the system is able to perform wireless monitoring and wireless control through Bluetooth communication.

TASK 1: BLUETOOTH IN IOT APPLICATIONS

Bluetooth can be incorporated into Internet of Things applications as a short-range wireless communication method between sensors, controllers, actuators, and user devices. In many IoT systems, a microcontroller collects data from the physical environment and sends the information to another device for monitoring or decision-making. Bluetooth is useful when the system does not need long-range communication and when the user is located near the device.

One example is a smart home monitoring system. Sensors can measure temperature, humidity, light intensity, or motion, and the data can be sent to a smartphone through Bluetooth. The user can view the condition of the room and send commands to switch devices on or off. Another example is a wearable health device, where data such as heart rate or movement can be transmitted to a mobile phone for display and logging.

Bluetooth is also suitable for industrial and laboratory applications because it can reduce cable usage during testing and troubleshooting. A technician can monitor machine parameters from a mobile device and send simple commands to adjust the system. In mechatronics, Bluetooth can be used to control mobile robots, servo mechanisms, grippers, or small automation modules. Therefore, Bluetooth supports IoT by enabling wireless sensing, local control, real-time data display, and convenient device-to-device communication.

TASK 2: SENSOR MONITORING AND SERVO CONTROL

For the Week 9 task, the original setup was modified so that it includes one sensor and one servo motor. The sensor provides the monitoring function, while the servo motor provides the control function. The smartphone terminal is used as the user interface for both receiving sensor data and sending servo control commands.

The ESP32 reads the sensor value and sends it to the smartphone at a fixed interval. The terminal application displays the received value, allowing the user to observe changes in real time. At the same time, the user can type a command into the terminal. The ESP32 reads the received command and maps it to a servo angle. For example, one command can move the servo to 0 degrees, another command can move it to 90 degrees, and another command can move it to 180 degrees.

This setup demonstrates a practical remote-control system because the user can monitor the sensor condition and adjust the actuator without directly touching the circuit. The task also shows that Bluetooth communication is bidirectional. Data is not only sent from the ESP32 to the smartphone, but commands are also sent from the smartphone to the ESP32. This is an important concept in IoT and automation because many systems require both feedback and control.

DISCUSSION

The experiment shows that Bluetooth communication can replace a wired serial connection for short-range monitoring and control. The smartphone terminal acted as a simple human-machine interface by displaying sensor data and allowing the user to send commands. This made the system easier to operate because the user did not need to keep the ESP32 connected to the Arduino IDE Serial Monitor after programming.

The sensor reading displayed on the smartphone depends on the type and condition of the sensor used. If a temperature sensor is used, the reading may change slowly according to room temperature. If a potentiometer or analog sensor is used, the reading may change immediately when the input is adjusted. Regardless of the sensor type, the important result is that the ESP32 successfully reads the value and transmits it through Bluetooth.

The servo motor response depends on correct wiring, stable power supply, and correct command interpretation. A servo may become unstable if it is powered directly from the ESP32 when the current demand is too high. Therefore, an external power supply is recommended for the servo if the movement becomes weak or unstable. The Bluetooth connection must also remain paired and connected during testing to prevent loss of command transmission.

Several limitations were identified. Bluetooth communication is suitable for short-range applications but is not ideal for long-distance monitoring. In addition, the system depends on a paired smartphone and a compatible terminal application. Interference from other 2.4 GHz devices may also affect connection quality. However, for educational mechatronics experiments and local IoT prototypes, Bluetooth is reliable, low-cost, and simple to implement.

CONCLUSION

In conclusion, the Week 9 experiment successfully demonstrates the concept of remote monitoring and control using Bluetooth communication with an ESP32. The ESP32 can transmit sensor readings wirelessly to a smartphone terminal application, allowing the user to monitor data without a USB serial connection. The smartphone can also send commands back to the ESP32 to control a servo motor.

The experiment proves that Bluetooth communication can be used as a simple IoT interface for mechatronic applications. By combining a sensor, an ESP32, a Bluetooth terminal, and a servo motor, the system performs both sensing and actuation. This provides a practical example of how wireless communication can improve flexibility, convenience, and user interaction in embedded control systems.

RECOMMENDATIONS

- Use a stable power supply for the servo motor, especially if the servo requires more current than the ESP32 can provide.
- Connect all grounds together to ensure that the ESP32, sensor, and servo share the same reference voltage.

- Use clear and simple command characters so that the ESP32 can interpret the smartphone input correctly.
- Test the Bluetooth connection before recording data to ensure that the smartphone is properly paired and connected.
- Add labels to the terminal output so that sensor readings are easy to understand.
- For future improvement, the system can be expanded by adding more sensors, data logging, or multiple actuator controls.

REFERENCES

- [1] Arduino Project Hub. Arduino Bluetooth Basic Tutorial.
- [2] How To Mechatronics. Arduino and HC-05 Bluetooth Module Complete Tutorial.
- [3] Arduino Project Hub. Basic Bluetooth Communication with Arduino and HC-05.
- [4] Espressif Systems. ESP32 Technical Reference and Bluetooth Overview.
- [5] Arduino Documentation. Servo Motor Control and Sensor Interfacing.

APPENDICES

Appendix A: Program Listing

```

1  #include "BluetoothSerial.h"
2  #include <ESP32Servo.h>
3  #include "DHT.h"
4
5
6  #define DHTPIN 15    // DHT22 Data pin connected to D15
7  #define DHTTYPE DHT22
8  const int servoPin = 13; // Servo signal pin connected to D13
9
10 BluetoothSerial SerialBT;
11 Servo myServo;
12 DHT dht(DHTPIN, DHTTYPE);
13
14 char command;
15 unsigned long previousMillis = 0;
16 const long interval = 2000; // 2 second interval for DHT22
17
18 void setup() {
19
20     Serial.begin(9600);
21
22     SerialBT.begin("Group 8_MSI");
23
24     dht.begin();
25
26     // Setup Servo
27     myServo.attach(servoPin);
28     myServo.write(0); // Start at home position
29
30
31
32     Serial.println("System Initialized.");
33     Serial.println("Bluetooth: Group 8_MSI | Sensor: D15 | Servo: D13");
34 }
35
36 void loop() {
37     // 1. BLUETOOTH COMMAND LISTENER (Instant Response)
38     if (SerialBT.available()) {
39         command = SerialBT.read();
40
41         Serial.print("Command received: ");
42         Serial.println(command);
43
44         switch (command) {
45             case 'A':
46                 myServo.write(0);
47                 SerialBT.println("Action: Servo moved to 0°");
48                 break;
49             case 'B':
50                 myServo.write(90);
51                 SerialBT.println("Action: Servo moved to 90°");
52                 break;
53             case 'C':
54                 myServo.write(180);
55                 SerialBT.println("Action: Servo moved to 180°");
56                 break;
57             default:
58                 break;
59         }
60     }
61
62     // 2. SENSOR MONITORING (Runs every 2 seconds)
63     unsigned long currentMillis = millis();
64
65     if (currentMillis - previousMillis >= interval) {
66         previousMillis = currentMillis;
67
68         float h = dht.readHumidity();
69         float t = dht.readTemperature();
70
71         if (isnan(h) || isnan(t)) {
72             Serial.println("Sensor Error: Check wiring at D15!");
73             SerialBT.println("Error: Sensor not responding.");
74         } else {
75             // Send to phone
76             SerialBT.print("Temp: ");
77             SerialBT.print(t);
78             SerialBT.print("C | Hum: ");
79             SerialBT.print(h);
80             SerialBT.println("%");
81
82             // Print to Laptop Serial Monitor
83             Serial.print("Temp: ");
84             Serial.print(t);
85             Serial.print("C | Hum: ");
86             Serial.print(h);
87             Serial.println("%");
88         }
89     }
90 }

```

Appendix B: Experimental Figures



Appendix C: Acknowledgement

We hereby declare that the work presented in this technical report is entirely our own, except where explicitly acknowledged. We certify that in developing and completing this mechatronics project, we have strictly adhered to the standards of academic integrity and have not engaged in any form of plagiarism or unethical conduct. All sources of information, reference materials, and technical support utilized in this work have been appropriately cited and acknowledged.

Appendix C: Declaration

Certificate of Originality and Authenticity

This is to certify that we are responsible for the work submitted in this report, that the original work is our own except as specified in the references and acknowledgment, and that the original work contained herein has not been untaken or done by unspecified sources or persons. We hereby certify that this report has not been done by only one individual, and all of us have contributed to the report. The length of contribution to the reports by each individual is noted within this certificate. We also hereby certify that we have read and understand the content of the total report, and no further improvement on the reports is needed from any of the individual contributors to the report. We therefore agreed unanimously that this report shall be submitted for marking, and this final printed report has been verified by us.

Signature: IrfanHakimi

Name: Muhammad Irfan Hakimi Bin Mohd Rizuan
Matric : 2416289

Read[/
Understand[/
Agree[/]

Signature: Najmi

Name: Muhammad Najmi Bin Mohd Azam
Matric : 2128653

Read[/
Understand[/
Agree[/]

Signature: Shahabudin

Name: Muhammad Shahabudin bin Ishak
Matric : 2120279

Read[/
Understand[/
Agree[/]